

SERVERLESS FUNCTION ASSIGNMENT – 8

IoT Sensor Data Processor

AWS Lambda + Amazon DynamoDB + Amazon SNS

Student Name: Thrupthi .J

Student ID	24UG00238
Assignment	8 – IoT Sensor Data Processor
Region	Asia Pacific (Mumbai) – ap-south-1
Platform	AWS CLI on Windows Command Prompt

1. Objective

The objective is to implement an IoT sensor data processor that collects simulated temperature and humidity readings, stores the readings in a database, checks threshold conditions, and sends an alert when a threshold is breached.

2. Assignment Requirement

Assignment 8 is the IoT Sensor Data Processor. It requires IoT sensor data such as temperature and humidity to be collected, stored in a database, and alerts to be generated when threshold values are exceeded.

3. AWS Architecture

Simulated Sensor Data (event.json) → AWS Lambda (IoTSensorProcessor) → DynamoDB (IoTSensorData) and, when a threshold is breached, Amazon SNS (IoTSensorAlerts) → Email notification.

4. AWS Resources Created

Service	Resource	Purpose
DynamoDB	IoTSensorData	Stores sensor readings and alert status.
Lambda	IoTSensorProcessor	Processes readings and checks thresholds.
IAM	IoTSensorLambdaRole	Execution role used by Lambda.
SNS	IoTSensorAlerts	Publishes threshold-breach notifications.
SNS subscription	Confirmed email endpoint	Receives alert notifications.

5. Threshold Logic

Temperature threshold: greater than 30 °C. Humidity threshold: greater than 70%. The demonstration used 35 °C and 80%, producing High Temperature, High Humidity.

6. Implementation – Step by Step

6.1 Verify AWS CLI authentication

Run the AWS STS identity command to confirm that the AWS CLI is connected to the account.

6.2 Create DynamoDB table

Create IoTSensorData with sensor_id as the partition key and timestamp as the sort key. PAY_PER_REQUEST billing was used.

6.3 Create Lambda execution role

Create IoTSensorLambdaRole with a trust policy allowing the Lambda service to assume the role.

6.4 Attach permissions

Attach CloudWatch logging permission and DynamoDB access to the Lambda execution role.

6.5 Create the Python Lambda function

The function reads sensor_id, temperature and humidity; checks thresholds; writes an item to DynamoDB; and publishes an SNS message if an alert exists.

6.6 Package and deploy

Save lambda_function.py, create lambda_function.zip, and deploy/update the Lambda function.

6.7 Create SNS topic and email subscription

Create IoTSensorAlerts, subscribe an email endpoint, and confirm the subscription from the AWS SNS confirmation email.

6.8 Test the serverless function

Use event.json with sensor_id 24UG00238, temperature 35 and humidity 80, then invoke the Lambda.

6.9 Verify results

Check DynamoDB for the stored record and verify that the threshold alert email was received.

7. Test Input

```
{"sensor_id": "24UG00238", "temperature": 35, "humidity": 80}
```

8. Lambda Python Code

```
import json
import boto3
from decimal import Decimal
from datetime import datetime, timezone

dynamodb = boto3.resource("dynamodb", region_name="ap-south-1")
table = dynamodb.Table("IoTSensorData")
sns = boto3.client("sns", region_name="ap-south-1")
SNS_TOPIC_ARN = "arn:aws:sns:ap-south-1:ACCOUNT_ID:IoTSensorAlerts"
TEMPERATURE_LIMIT = 30
HUMIDITY_LIMIT = 70

def lambda_handler(event, context):
    sensor_id = event.get("sensor_id", "24UG00238")
    temperature = Decimal(str(event.get("temperature", 0)))
    humidity = Decimal(str(event.get("humidity", 0)))
    alerts = []
    if temperature > TEMPERATURE_LIMIT:
        alerts.append("High Temperature")
    if humidity > HUMIDITY_LIMIT:
        alerts.append("High Humidity")
    alert = ", ".join(alerts) if alerts else "Normal"
    timestamp = datetime.now(timezone.utc).isoformat()
    item = {"sensor_id": sensor_id, "timestamp": timestamp,
           "temperature": temperature, "humidity": humidity, "alert": alert}
    table.put_item(Item=item)
    if alerts:
        message = ("IoT Sensor Alert\n\n"
                  f"Sensor ID: {sensor_id}\n"
                  f"Temperature: {temperature} °C\n"
                  f"Humidity: {humidity} %\n"
                  f"Alert: {alert}\n"
                  f"Time: {timestamp}")
        sns.publish(TopicArn=SNS_TOPIC_ARN,
                   Subject="IoT Sensor Threshold Alert",
                   Message=message)
    return {"statusCode": 200,
            "body": json.dumps({"sensor_id": sensor_id,
                               "temperature": float(temperature),
                               "humidity": float(humidity),
                               "alert": alert})}
```

9. Results

The Lambda invocation returned StatusCode 200 with no FunctionError. DynamoDB contained sensor records with sensor_id 24UG00238, temperature 35, humidity 80 and alert High Temperature, High Humidity. An SNS threshold alert email was successfully received.

10. Screenshot Evidence

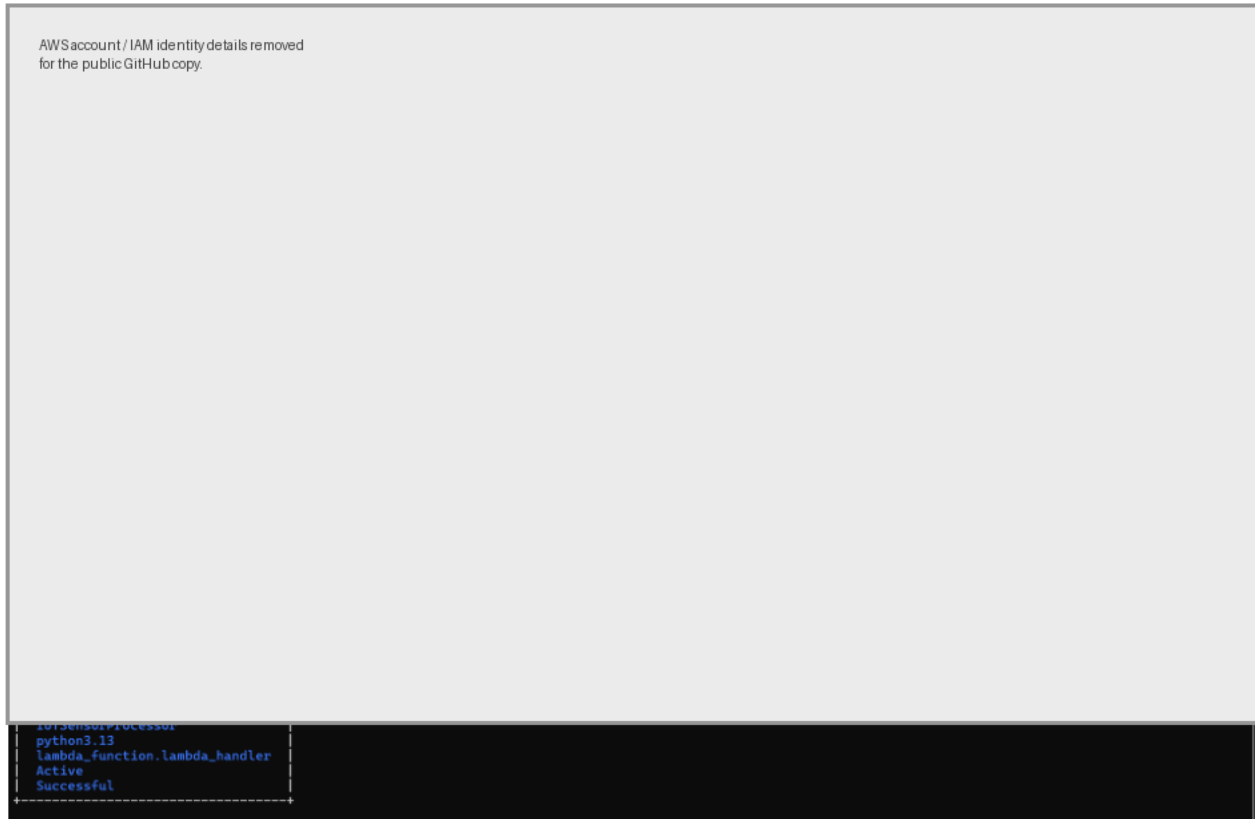


Figure 3 – AWS Lambda IoTSensorProcessor configuration.

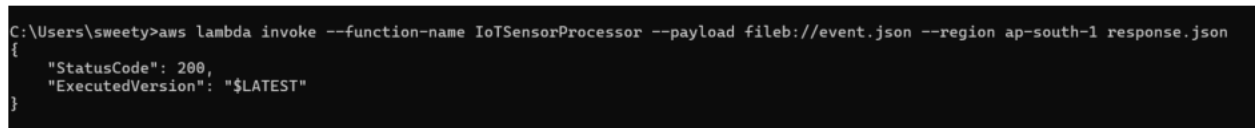


Figure 4 – Successful Lambda invocation with StatusCode 200.

```
C:\Users\sweet>aws dynamodb scan --table-name IoTSensorData --region ap-south-1
{
  "Items": [
    {
      "humidity": {
        "N": "88"
      },
      "alert": {
        "S": "High Temperature, High Humidity"
      },
      "sensor_id": {
        "S": "24UG00238"
      },
      "temperature": {
        "N": "35"
      },
      "timestamp": {
        "S": "2026-09-10T05:08:56.668004+00:00"
      }
    },
    {
      "humidity": {
        "N": "88"
      },
      "temperature": {
        "N": "35"
      },
      "sensor_id": {
        "S": "24UG00238"
      },
      "alert": {
        "S": "High Temperature, High Humidity"
      },
      "timestamp": {
        "S": "2026-09-10T05:23:29.715536+00:00"
      }
    },
    {
      "humidity": {
        "N": "88"
      },
      "sensor_id": {
        "S": "24UG00238"
      }
    }
  ]
}
```

Figure 5 – Sensor records stored in DynamoDB, including the threshold alert.

Figure 6 Note

The uploaded SNS CLI screenshot shows an earlier subscription record with status 'Deleted'. It should not be used as final proof of the confirmed subscription. The subscription was subsequently confirmed and the successful email in Figure 7 proves SNS delivery worked. For final submission, capture a fresh list-subscriptions-by-topic screenshot after confirmation.

SNS subscription/account details removed for the public GitHub copy.

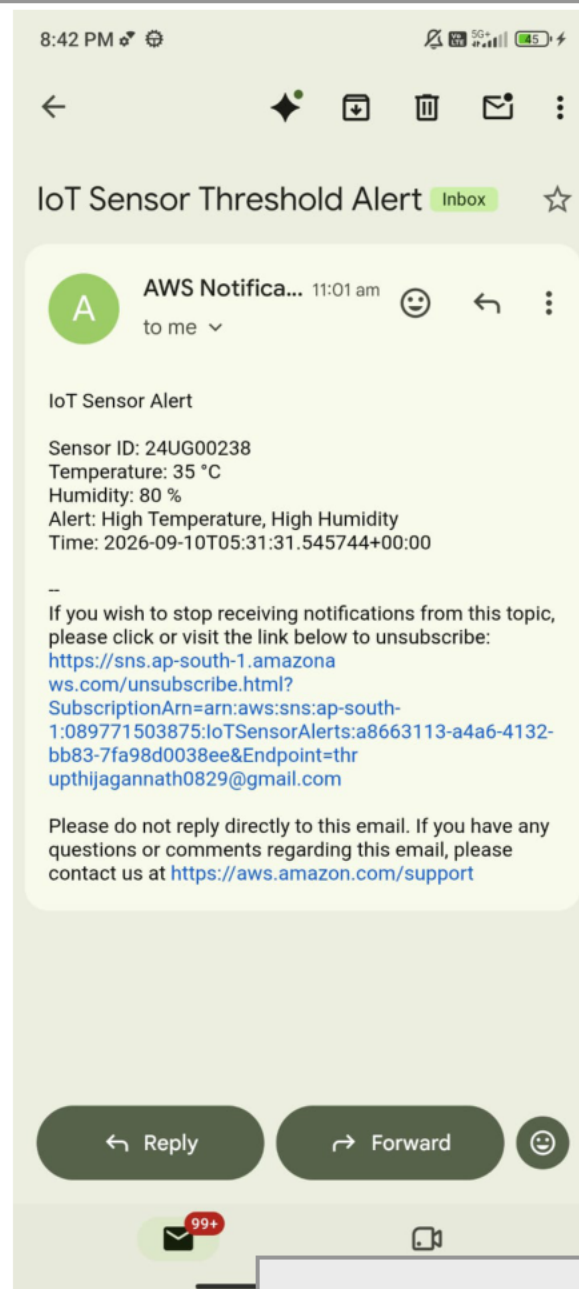


Figure 7 – Receive

SNS subscription URL removed

11. Conclusion

The serverless IoT sensor data processor was successfully implemented on AWS. The system accepts simulated sensor readings, processes them through AWS Lambda, stores the readings in DynamoDB, detects threshold breaches, and sends an email notification through Amazon SNS.

12. Final Submission Checklist

- Include AWS CLI identity verification.
- Include DynamoDB table status and sensor-record evidence.
- Include Lambda configuration and successful execution.
- Replace the historical SNS screenshot with a fresh confirmed subscription screenshot.
- Include the received threshold alert email.
- Include the Python code and test input.